

中山大学专业课《程序设计I》

第十节 函数与指针

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

本节目标

1. 掌握const用法

- 限定原则
- 转换规则

2. 掌握函数指针

- 函数指针声明
- 利用函数指针调用函数
- 回调函数

3. 学会动态分配内存

- malloc()
- free()
- memcpy()

思考一下

以下代码哪行会报错？

```
int x = 10;  
const int *p1 = &x;  
int * const p2 = &x;
```

*p1 = 20;	// A	✗
p1 = NULL;	// B	✓
*p2 = 20;	// C	✓
p2 = NULL;	// D	✗

const 修饰符

❑ **const** 限定类型对象的左值表达式，表示不可修改左值，即不可赋值

❑ 原则：const 限定右侧的整个左值表达式

■ **const** int x $\equiv$ int **const** x

x = 20;



■ **const** int *p $\equiv$ int **const** *p

*p = 20;



p = NULL;



■ int * **const** p

*p = 20;



p = NULL;



■ **const** int * **const** *p

const 修饰符

❑ **const** 限定类型对象的左值表达式，表示不可修改左值，即不可赋值

❑ 原则：const 限定紧邻右侧的整个左值表达式

■ **const** int x $\equiv$ int **const** x

x = 20; ❌

■ **const** int *p $\equiv$ int **const** *p

*p = 20; ❌

p = NULL; ✅

■ int * **const** p

*p = 20; ✅

p = NULL; ❌

■ **const** int * **const** *p

*p = NULL; ❌

p = NULL; ✅

const 修饰符

❑ **const** 限定类型对象的左值表达式，表示不可修改左值，即不可赋值

❑ 原则：const 限定紧邻右侧的整个表达式

■ **const** int x $\equiv$ int **const** x

x = 20;



■ **const** int *p $\equiv$ int **const** *p

*p = 20;



p = NULL;



■ int * **const** p

*p = 20;



p = NULL;



■ **const** int * const *p $\equiv$ int **const** * const *p

*p = NULL;



p = NULL;



**p = 20;



练习

哪些语句会报错?

```
const int * const * const p;
```

```
p = NULL;           // A
```

```
*p = NULL;          // B
```

```
**p1 = 20;           // C
```

```
int x = 5;
```

```
int const *p1 = &x;
```

```
const int **p2 = &p1;
```

```
p2 = NULL;           // A
```

```
*p2 = NULL;          // B
```

```
**p2 = 20;            // C
```

```
x = 20;               // D
```

const 与非const 之间的转换

❑ const 限定的左值表达式**只能初始化，不能赋值**

❑ 指向常量指针和普通指针间的转换

■ **非const** 可隐式转换为 const

■ const 赋予 **非const** 必须显式转型

```
const int n = 1; // 正确: 可以初始化  
n = 2; // 错误: 不可以赋值
```

```
int *p1 = NULL;  
const int *p2 = p1; // 正确: 非const转const  
p1 = p2; // 错误: 舍弃限定符  
p1 = (int *)p2; // 正确: 显式转型
```

❑ 在函数**形参声明**中

■ const double x[] 等价 const double *x

■ double x[const] 等价 double * const x

这两位函数是.....?

```
void SelectedSortInt(int *nums, size_t n) {  
    for (int i=0; i<=n-2; i++) {  
        // find minimum's idx  
        int idx = i;  
        for (int j=i+1; j<=n-1; j++)  
            if (nums[j] < nums[idx])  
                idx = j;  
        // swap  
        if (i != idx) {  
            int temp = nums[i];  
            nums[i] = nums[idx];  
            nums[idx] = temp;  
        }  
    }  
}
```

选择排序

```
void bubbleSort(int arr[], int n) {  
    int i, j, temp;  
    for (i=0; i<n-1; i++)  
        for (j=0; j<n-1-i; j++)  
            // 如果前面的元素比后面的大, 就交换  
            if (arr[j] > arr[j+1]) {  
                temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
}
```

冒泡排序

C语言 stdlib.h 自带的排序函数——qsort

```
void qsort(void *arr, size_t lens, size_t n, int (*compar)(const void *, const void *));
```

数组名 数组长度 元素大小 函数指针
(首元素指针) (元素个数) (字节) (函数指针? ? ? 先不管, 一会再说)

void *: 无类型指针 (注意不是空指针)

- 指针变量仅保存指针的值 (地址值), 忽略指向数据的类型 (不能解引用)
- 任意类型指针都可以隐式转为 void *
- 这意味着 qsort 可以接收任意类型的地址值

C语言自带的排序函数——qsort

```
#include <stdio.h>
#include <stdlib.h>

int compfun(const void *p1, const void *p2){
    int *pr1 = (int *)p1;
    int *pr2 = (int *)p2;
    return *pr1 - *pr2;
}
```

```
int main(void){
    int arr[10] = {8,4,5,3,7,1,2,9,6,0};
    qsort(arr, 10, sizeof(arr[0]), compfun);
    for(int i=0; i<10; i++)
        printf("arr[%d]=%d\n", i, arr[i]);
}
```

compfun是整数比较函数，用于比较两个整数元素。
qsort要求比较函数返回一个整数，表示比较结果：

小于零：表示第一个元素 **小于** 第二个元素

等于零：表示第一个元素 **等于** 第二个元素

大于零：表示第一个元素 **大于** 第二个元素

函数指针允许我们在一个函数中有选择地调用其他函数！

这里传进来的其实是**compfun**函数的指针（地址）

函数指针

- ❑ **函数地址**：程序中定义了一个函数，程序加载时系统会为此函数代码分配一段内存空间，这段存储空间的**首地址**称为这个函数的地址

- 函数名表示的就是这个地址

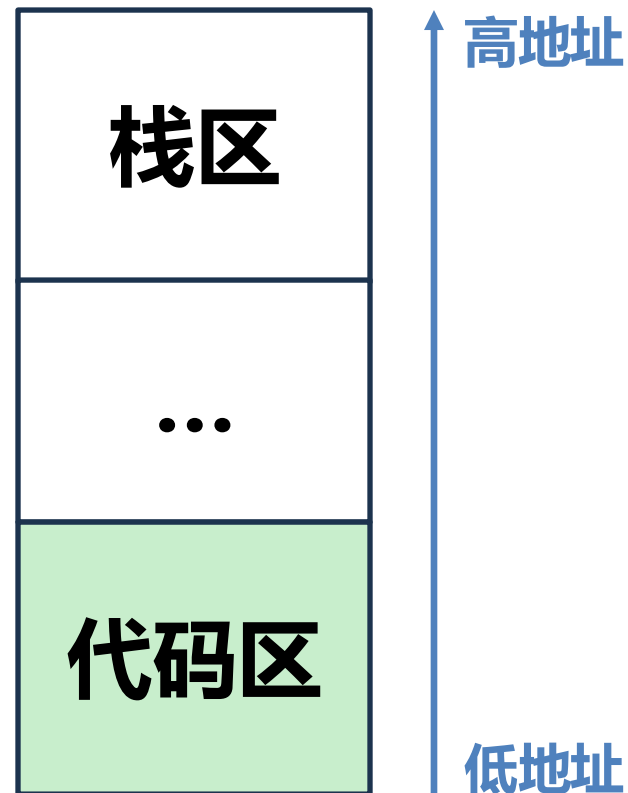
- ❑ **函数指针**：存放函数地址的指针变量

- ❑ **函数指针声明**：`返回值类型 (*变量名) (参数类型列表);`

函数声明：`int max(int, int);`

函数指针定义：`int (*p)(int, int);` p可指向所有返回值为int，参数为(int,int)的函数

- ❑ **注意**：函数指针变量**没有算术运算、下标运算**



用函数指针调用函数

```
#include <stdio.h>
```

```
int max(int a, int b){  
    return a>b?a:b;  
}
```

声明函数，max为函数名，值是函数首地址

```
int main(void){
```

```
    int (*p)(int,int);
```

声明函数指针

```
    p = max;
```

将max函数的首地址赋值给p

```
    printf("%d\n", (*p)(10,20));
```

用函数指针调用函数

```
}
```

回调函数

函数指针允许我们将函数指针作为函数参数，由函数外部逻辑决定函数行为

```
void qsort(void *arr, size_t lens, size_t n, int (*compar)(const void *, const void *));
```

函数指针形参

```
qsort(arr, 10, sizeof(arr[0]), compfun);
```

回调函数

如何将选择排序改成回调函数版本?

```
void SelectedSortInt(int *nums, size_t n) {  
    for (int i=0; i<=n-2; i++) {  
        // find minimum's idx  
        int idx = i;  
        for (int j=i+1; j<=n-1; j++)  
            if (nums[j] < nums[idx])  
                idx = j;  
        // swap  
        if (i != idx) {  
            int temp = nums[i];  
            nums[i] = nums[idx];  
            nums[idx] = temp;  
        }  
    }  
}
```

```
void SelectSort(void *nums, size_t n, size_t m,  
int (*compar)(const void *, const void *))
```

与qsort一样, 需要支持一般数组

```
nums[j]          nums[idx]  
(*compar)((char *)nums+j*m, (char *)nums+idx*m) < 0
```

利用比较函数比较

```
void *temp = malloc(m);  
memcpy(temp, nums+i*m, m);  
memcpy(nums+i*m, nums+idx*m, m);  
memcpy(nums+idx*m, temp, m);  
free(temp);
```

malloc? free? memcpy?

不知道num[i]的类型...

动态分配内存

- ❑ C语言允许我们在程序运行时用 `stdlib.h` 中的 `malloc()` 函数动态分配内存

```
void * malloc(size_t size);
```

返回该内存首地址

所需内存字节数

- ❑ `malloc()` 要与 `free()` 配套使用，释放不再使用的内存，否则可能导致内存耗尽（内存溢出）

```
void free(void* ptr);
```

`malloc`创建的动态内存地址

内存拷贝

```
void *memcpy(void *p1, const void *p2, size_t n)
```

```
void SelectedSortInt(int *nums, size_t n) {  
    for (int i=0; i<=n-2; i++) {  
        // find minimum's idx  
        int idx = i;  
        for (int j=i+1; j<=n-1; j++)  
            if (nums[j]<nums[idx])  
                idx = j;  
        // swap  
        if (i != idx) {  
            int temp = nums[i];  
            nums[i] = nums[idx];  
            nums[idx] = temp;  
        }  
    }  
}
```

有了malloc()分配的内存，我们还需要一种方法把nums[i]的值拷贝到动态内存中

string.h 中的 `memcpy(p1,p2,n)` 可以从存储区 `p2` 拷贝 `n` 个字节到存储区 `p1`

如何将选择排序改成回调函数版本？

```
void SelectedSortInt(int *nums, size_t n) {  
    for (int i=0; i<=n-2; i++) {  
        // find minimum's idx  
        int idx = i;  
        for (int j=i+1; j<=n-1; j++)  
            if (nums[j] < nums[idx])  
                idx = j;  
        // swap  
        if (i != idx) {  
            int temp = nums[i];  
            nums[i] = nums[idx];  
            nums[idx] = temp;  
        }  
    }  
}
```

```
void SelectSort(void *nums, size_t n, size_t m, int  
(*compar)(const void *, const void *))
```

与qsort一样，需要支持一般数组

nums[j] nums[idx]

```
(*compar)((char *)nums+j*m, (char *)nums+idx*m) < 0
```

利用比较函数比较

```
void *temp = malloc(m);
```

分配m字节的内存

```
memcpy(temp, nums+i*m, m);  
memcpy(nums+i*m, nums+idx*m, m);  
memcpy(nums+idx*m, temp, m);
```

交换i和idx

```
free(temp);
```

释放分配的内存

练习1：推测函数功能，并于课后自行验证

```
void* MemRepeat(void *dest, void *src,
size_t src_sz, size_t count) {
    void *p=dest;
    for (int i=0; i<count; i++) {
        memcpy(p, src, src_sz);
        p += src_sz;
    }
    return dest;
}

void Printx(void *p, size_t bytes_count) {
    for (int i=0; i<bytes_count; i++,p++)
        printf("%4x", *(unsigned char*)p);
    printf("\n");
}
```

补充：字符串字面量和字符串函数

字符串字面量

- ❑ 字符串字面量在程序执行前会被加载到**内存的只读区**
- ❑ 相同的字符串字面量只有一份

```
char *p1 = "Coffe";  
char *p2 = "Coffe";  
printf("p1 = %p\n", p1);  
printf("p2 = %p\n", p2);  
  
*p1 = 'H'; // 错误
```

思考

以下代码有什么问题？ 会打印什么？

```
char *ss[] = {"hello", "hello", "!"};  
ss[0][0] = 'H';  
char sa[][10] = {"hello", "hello", "!"};  
sa[0][0] = 'H';  
  
printf("%s\n", *ss);  
printf("%s\n", *sa);  
printf("%d\n", ss[0] == ss[1]);
```

字符串函数拓展

头文件	分类	常见函数 (红色需掌握)
ctype.h	字符分类	isalnum, isalpha, isdigit, islower, isupper, isspace, isprint
	字符操作	tolower, toupper
stdlib.h	转换成数值	atof, atoi (注: itoa 是非标函数)
string.h	字符串操作	strcpy, strncpy, strcat, strncat
	字符串检验	strlen, strcmp, strchr, strrchr, strstr
	字符数组操作	memchr, memcmp, memset, memcpy, memmove
	杂项	strerror

练习2：今天星期几？请完善以下代码

```
/*weekday*/
const char* weekday(int year, int month, int day) {
    const char *weekdays[] = {"Sunday",
                                "Monday", "Tuesday", "Wednesday",
                                "Thursday", "Friday", "Saturday"};
    static int m[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
    m[1] = leap(year) ? 29 : 28;
    int sum_days = day;
    for (int i=0,*p=m; i<month-1; i++)
        sum_days += *(p++);
    for (int i=1; i<year; i++)
        sum_days += leap(i) ? 366 : 365;
    return weekdays[sum_days % 7];
}
```

谢谢！

邮箱: yuesh5@mail.sysu.edu.cn